HyperQ: GPU-Accelerated Hyperdimensional Computing with Drift-Aware Stabilization for Noise-Robust Inference

Samarth Kale
Computer Engineering Dept.
FCRIT, Navi Mumbai, India
samarthnkale@gmail.com

Abhinaya Gowda
Computer Engineering Dept.
FCRIT, Navi Mumbai, India
gowdaabhinaya9@gmail.com

Dylan Braganza
Computer Engineering Dept.
FCRIT, Navi Mumbai, India
dylan.brag06@gmail.com

Keith Dias
Computer Engineering Dept.
FCRIT, Navi Mumbai, India
keithdias07@gmail.com

***Abstract*—Hyperdimensional Computing (HDC) classifiers are known to struggle in two practically important scenarios that standard benchmarks rarely capture: noisy input signals and gradual prototype drift during long-running deployment. This paper presents HyperQ, a GPU-based HDC framework that jointly addresses both challenges through hypersphere-constrained encoding, a transformer-based drift detection mechanism, and reinforcement-learning-guided adaptive precision control. When evaluated under significant input corruption, HyperQ demonstrates substantially higher retained accuracy compared to conventional neural baselines, even when more than half of the signal quality is degraded. Similarly, under sustained prototype drift over many inference iterations, HyperQ maintains functionally usable accuracy while an unprotected baseline collapses to near-random performance. Prototype quantization to INT8 achieves a fourfold reduction in memory footprint with no measurable loss in classification performance. These results suggest that HyperQ is a practical and deployable framework for edge AI scenarios where clean, stable data cannot be assumed.**

***Index Terms*—Hyperdimensional Computing, Vector Symbolic Architectures, Noise Robustness, Prototype Drift, Transformer Drift Detection, Reinforcement Learning, Edge AI, GPU Acceleration, Mixed Precision**

I. Introduction

Real-world deployments rarely look like benchmark evaluation sets. Autonomous mobile robots pick up broadband sensor noise from vibration and electromagnetic interference [4]. On-board spacecraft classifiers operate under cosmic-ray-induced bit-flip events that corrupt stored values [3]. Wearable biosensors drift over hours as electrode impedance shifts [5]. In each of these settings, the classifier must tolerate corrupted inputs, hold stable internal representations under deployment-time updates, or manage both at once.

Deep neural networks perform well on clean data but degrade sharply when these conditions arise. An MLP trained to high accuracy on MNIST drops to roughly half its clean performance under moderate Gaussian input noise; a CNN does only slightly better. Standard GPU-HDC without any drift correction deteriorates monotonically to near-random accuracy after several dozen inference iterations under cumulative prototype corruption.

Hyperdimensional Computing [1] takes a structurally different approach to classification. Data is encoded as bipolar hypervectors $\boldsymbol{v} \in \{-1, +1\}^D$ with $D \gg 1$, and classification proceeds by cosine similarity against class prototypes. The holographic, distributed nature of this representation [2] produces an inherent geometric noise floor: small perturbations in input space translate to proportionally smaller shifts in hypervector space. Standard HDC implementations fail to fully exploit this property, though, for two reasons. First, sign-aggregated prototype bundling ignores inter-class cosine margins, which limits the effective noise floor. Second, there is no closed-loop mechanism to correct prototype drift during long-running deployment.

HyperQ is a GPU pipeline that addresses both problems. Prototypes are learned via gradient-based margin optimization on the unit hypersphere, replacing the conventional sign-aggregated bundling step. A lightweight two-layer transformer continuously monitors prototype similarity evolution and triggers targeted re-encoding when drift crosses a threshold. A UCB1 bandit controller dynamically selects among FP32, FP16, and INT8 precision to balance accuracy and efficiency without manual tuning.

The contributions of this paper are as follows:

- A data-seeded, margin-optimizing cosine prototype learner that substantially reduces noise-induced accuracy degradation relative to MLP baselines (Section IV).
- A transformer-based Drift Detector with closed-loop re-encoding that preserves functionally acceptable accuracy under cumulative drift, where an unprotected baseline collapses entirely (Section IV).
- An RL-guided UCB1 Precision Controller achieving a fourfold prototype memory reduction with no accuracy penalty (Section IV).
- Reproducible T4-GPU experiments on MNIST and UCI HAR with all results traceable to a public Colab notebook (Section VI).

II. Literature Survey

A. Hyperdimensional Computing and Vector Symbolic Architectures

The concept of employing hypervectors for classification originated with Kanerva [1], who thought about memory in terms of high-dimensional random vector spaces. Then Plate [2] showed up and specified the algebraic framework of this concept through certain a thing called Holographic Lessened Representations. Kleyko et al. [3] looked at Vector Symbolic Architectures generally and they formally described how these systems can be nearly orthogonal and tolerate noise across different high-dimensional spaces. On the hardware side Imani et al. [4] used hypervector-based classification for classification on small devices that do not use a lot of power. Salamat et al. [5] took this idea further by using something called FPGA acceleration. They found that it made hypervector-based classification much more energy efficient. The class-hypervectors in modern hypervector-based classification systems comes from Rahimi et al. [9]. Gaylers earlier work [10] on reduced representations connected hypervector-based classification to distributed associative memory models, which is a way of thinking about how our brains work.

B. Noise Robustness in Machine Learning

HDC's theoretical noise tolerance comes from measure concentration in high-dimensional spaces [1]. Class prototypes separated by large cosine margins remain distinguishable under moderate additive perturbation. Poduval et al. [11] confirmed this through experiments, showing that HDC maintains accuracy even with substantial random bit-flip injections at sufficiently high dimensionality. Hernandez-Cano et al. [12] found that an adaptive, gradient-tuned encoder matrix consistently outperforms fixed RFF encoding for robust classification. Goodfellow et al. [13] characterized the vulnerability of deep networks to adversarial perturbations, which led to further work on structured attack methods. Madry et al. [14] formalized projected gradient descent for training models to be adversarially robust. They demonstrated how challenging it is to defend against structured attacks without explicitly augmenting training data.

C. Concept Drift Detection

Bifet and Gavalda [7] developed statistical drift detection for data streams, using ADWIN as a reference method for adaptive windowing. Their method is reactive, meaning it confirms drift only after accuracy has already declined. Since then, transformer-based sequence models have demonstrated effective temporal pattern recognition in online learning environments [8]. HyperQ uses a transformer on the time series of prototype cosine-similarity snapshots. This allows for proactive drift prediction before accuracy drops, instead of doing so after.

D. GPU Acceleration and Mixed Precision

Mixed-precision training, as described by Micikevicius et al. [6], set up a practical framework for FP16 and INT8 computation in deep learning pipelines. HyperQ modifies this approach for prototype storage. Courbariaux et al. [18] demonstrated that extreme quantization can maintain usable accuracy with binarized neural networks when the representations fit well structurally. Jacob et al. [19] built integer-only inference pipelines for neural networks using symmetric quantization, the same way that HyperQ employs for hypervector prototypes.

III. Mathematical Framework

A hypervector is a bipolar binary vector of dimension D:

$$\boldsymbol{v} \in \{-1, +1\}^D, \quad D \gg 1. \tag{1}$$

Continuous inputs $\boldsymbol{x} \in \mathbb{R}^d$ are mapped to the unit hypersphere using Random Fourier Features [6] with weight matrix $W \in \mathbb{R}^{d \times D}$, $W_{ij} \sim \mathcal{N}(0,\, \sigma^2)$, $\sigma = 0.05$:

$$\boldsymbol{h} = \frac{\cos\big(W^T \boldsymbol{x} + \boldsymbol{b}\big)}{\|\cos(W^T \boldsymbol{x} + \boldsymbol{b})\|_2}, \quad \boldsymbol{b} \sim \mathcal{U}[0,\, 2\pi]^D. \tag{2}$$

Projecting onto the unit hypersphere normalizes magnitude, which makes the subsequent cosine similarity inherently stable to symmetric additive noise. A perturbation $\boldsymbol{\epsilon}$ shifts the encoded point by $\|\boldsymbol{\epsilon}\|_2/D$ in cosine distance on average, an effect that diminishes as D grows.

Class prototypes $\{\boldsymbol{C}_k\}_{k=1}^K$ are initialized at normalized class centroids and refined via a temperature-scaled cosine loss:

$$\mathcal{L} = \text{CrossEntropy}\left(\frac{\boldsymbol{H}\,\tilde{\boldsymbol{C}}^T}{\|\tilde{\boldsymbol{C}}\|_2} \cdot T,\ \boldsymbol{y}\right), \quad T = 30, \tag{3}$$

where $\tilde{\boldsymbol{C}} = \boldsymbol{C}/\|\boldsymbol{C}\|_2$ is the row-normalized prototype matrix, re-applied after each Adam step to enforce the hypersphere constraint.

Inference uses cosine similarity:

$$\hat{y} = \arg\max_k\ \boldsymbol{h} \cdot \boldsymbol{C}_k \,/\, D. \tag{4}$$

A margin-based confidence score drives re-encoding decisions:

$$\text{conf}(\boldsymbol{h}) = \frac{s_{\max} - s_{\text{second}}}{s_{\max} + \epsilon}, \quad \epsilon = 10^{-6}. \tag{5}$$

The RL reward for precision arm a_t at step t is:

$$r_t = \alpha\, \text{acc}_t - \beta \|\Delta_t\|_2, \quad \alpha = 1.0,\ \beta = 0.01, \tag{6}$$

where $\Delta_t = \boldsymbol{C}(t) - \boldsymbol{C}(t{-}1)$ is the prototype change vector, penalizing precision choices that destabilize the prototype geometry.

IV. HyperQ Architecture

A. Overview

HyperQ has four modules in a closed-loop inference pipeline: (1) the Hypersphere RFF Encoder, (2) the Cosine Prototype Learner, (3) the Drift Detector, and (4) the Precision Controller. During inference, encoded hypervectors are compared with stored prototypes using cosine similarity. The Drift Detector checks prototype health in the background. If the drift probability exceeds the threshold of$\tau = 0.15$, a re-encoding pass over a cached representative batch is triggered without interrupting inference.

HyperQ has four modules in a closed-loop inference pipeline: (1) the Hypersphere RFF Encoder, (2) the Cosine Prototype Learner, (3) the Drift Detector, and (4) the Precision Controller. During inference, encoded hypervectors are compared with stored prototypes using cosine similarity. The Drift Detector checks prototype health in the background. If the drift probability exceeds the threshold of $\tau = 0.15$, a re-encoding pass over a cached representative batch is triggered without interrupting inference.

B. Hypersphere RFF Encoder

A low-variance Gaussian projection ($\sigma = 0.05$) keeps the local structure of the input manifold in the high-dimensional projection. This stops the cosine transform from reaching extreme values, which is the main problem with standard RFF encoders used on moderate-dimensional inputs like the 784-dimensional MNIST feature vectors. The ℓ_2-normalization step moves all hypervectors onto the unit sphere, making cosine similarity a geometrically scaled distance, no matter the input size.

C. Cosine Prototype Learner

Conventional hyperdimensional computing (HDC) prototype construction involves averaging encoded training samples for each class. A sign threshold is then applied to the averaged representations. Computationally efficient, but it ignores inter-class cosine margins entirely. HyperQ replaces it with a two-stage procedure. Stage 1 initializes each prototype at the ℓ_2-normalized centroid of its class in hypervector space, encoding global class geometry as a warm start. Stage 2 refines the prototype matrix for 25 epochs using the Adam optimizer ($\mathrm{lr} = 0.01$) with the temperature-scaled cosine loss (Eq. 3), re-normalizing after each step to maintain the hypersphere constraint. This procedure increases clean MNIST accuracy from 81.18% (Standard HDC baseline, $D = 10{,}000$) to 90.22% (Cosine-HDC, $D = 4096$) for geometric robustness evaluation (Table V), and to 94.16% (Full System, $D = 10{,}000$, INT8) for the peak efficiency configuration (Table III).

D. Drift Detector

A two-layer transformer encoder processes a sliding window of $L = 20$ prototype cosine-similarity snapshots $\boldsymbol{s}_t \in \mathbb{R}^K$. Each entry notes the average pairwise cosine similarity for class k. The transformer produces a scalar drift probability $p_{\mathrm{drift}} \in [0, 1]$. When $p_{\mathrm{drift}} > \tau = 0.15$, a complete re-encoding pass is initiated. The detector is pretrained on synthetic sequences. Stable windows (with within-window variance < 0.05) receive label 0, while drifting windows (with variance > 0.25) receive label 1. Thirty epochs of pretraining are enough (final binary cross-entropy < 0.05). After this, the detector weights are frozen for deployment.

E. Precision Controller

The Precision Controller dynamically selects between FP32, FP16, and INT8 computation using UCB1 [8] with optimistic initialization ($Q_0 = 0.9$ for all arms). The UCB1 index at step t for arm a is:

$$\mathrm{UCB}(a, t) = \hat{Q}(a) + \sqrt{\frac{2 \ln t}{N(a) + \epsilon}}. \tag{7}$$

INT8 uses symmetric quantization with scale $= \max |\boldsymbol{C}|/127$, clipping to $[-128, 127]$. After more than 1,000 inference steps on MNIST, the controller settles on FP16. On UCI HAR, it settles on INT8, which aligns with the precision trade-off results in Section VI.

F. Hardware Platform

All experiments ran on a Tesla T4 GPU (CUDA Compute Capability 7.5, 15.6 GB GDDR6, 320 Tensor Cores) in a Google Colab Pro environment with PyTorch 2.10 and CUDA 12.8. The T4 is typical of server GPUs used for inference in autonomous vehicle compute platforms and edge AI servers.

V. Experimental Setup

A. Datasets

MNIST. 60,000 training and 10,000 test grayscale digit images (28×28) [1]. Pixels are scaled to $[0, 1]$ and flattened to 784-dimensional vectors. Noise experiments apply only to the test set; training data stays clean so that results reflect genuine robustness rather than augmentation effects.

UCI HAR. The UCI Human Activity Recognition dataset [5] contains 7,352 training and 2,947 test samples of 561-dimensional smartphone accelerometer and gyroscope features across six activity classes. Features are standardized to zero mean and unit variance using training-set statistics.

B. Baselines

Standard HDC (Bundling): HDC with sign-aggregated prototypes and FP16 inference, no drift stabilization ($D = 10{,}000$). This configuration isolates the contribution of HyperQ's learning and stabilization procedures.

GPU-MLP: Three-layer perceptron (784–512–256–n_c), trained for 15 epochs using Adam ($\mathrm{lr} = 10^{-3}$) with cross-entropy loss.

GPU-CNN: Two-block convolutional network (2× Conv–MaxPool) with a two-layer classification head, trained for 15 epochs on MNIST only.

SVM (RBF): A scikit-learn SVM with RBF kernel, with C selected by 5-fold cross-validation, included as a non-GPU statistical reference.

TABLE I
Hardware Platform

Specification	Tesla T4
CUDA Compute Capability	7.5
CUDA Cores	2,560
Tensor Cores	320
VRAM	15.6 GB GDDR6
TDP	∼70 W
Framework	PyTorch 2.10 / CUDA 12.8

TABLE II
ABLATION STUDY CONFIGURATIONS (MNIST, $D = 10{,}000$)

Config	RFF	FP16	INT8	RL
Baseline	×	×	×	×
+RFF	✓	×	×	×
+FP16	✓	✓	×	×
+Packing	✓	✓	✓	×
+RL	✓	✓	✓	✓
HyperQ	✓	✓	✓	✓

C. Evaluation Metrics

Throughput (samples/s): Mean over 5 trials with 3 warm-up passes and CUDA synchronization barriers bounding each measurement.

Accuracy (%): Top-1 on the held-out test partition.

Energy (mJ/sample): Derived from NVML power readings sampled at the start and end of each pass: $E = \frac{P_0+P_1}{2} \cdot \Delta t \cdot \frac{1000}{N}$.

Noise Robustness: Accuracy drop when clean-trained models are evaluated on corrupted test inputs. Two regimes are reported: (a) moderate Gaussian noise $\mathcal{N}(0,\, 0.45^2)$ clipped to $[0, 1]$; and (b) a compound hard attack combining pixel inversion, strong Gaussian noise, 30% dropout, and 4-level coarse quantization.

Drift Stability: Accuracy tracked over 50 iterations under cumulative multiplicative decay ($\gamma = 0.98$) plus additive noise ($\sigma_d = 0.05$). HyperQ triggers re-encoding every 15 iterations; the baseline receives no correction.

Stability (σ): Standard deviation of accuracy across 10 independent seeds.

D. Ablation Configurations

Six incremental configurations isolate component contributions, all evaluated on MNIST with $D = 10{,}000$. See Table II.

VI. RESULTS AND ANALYSIS

A. Clean Inference Performance and Efficiency

Two model configurations are distinguished throughout this section to keep comparisons clean. **HyperQ (Full System)** runs at $D = 10{,}000$ using the complete pipeline – INT8 quantization, drift detection, and RL precision control – to maximize clean inference efficiency. **Cosine-HDC** isolates geometric robustness at $D = 4096$, simulating tighter edge hardware memory constraints.

Table III shows clean MNIST performance across all methods. HyperQ (Full System) achieves 94.16% accuracy in INT8 mode. The GPU-MLP (98.09%) and GPU-CNN (99.02%) reach higher peak accuracy on clean data, but HyperQ delivers a structural efficiency advantage that matters in practice. As shown in Table IV, HyperQ's prototype matrix uses $100{,}000$ trainable parameters (10 classes $\times$ 10,000 dimensions), fitting in just 0.095 MB at INT8 – compared to $535{,}818$ parameters for the MLP and roughly 1.8 M for the CNN. At $347{,}873$ samples/s and 0.203 mJ/sample, HyperQ runs nearly $3\times$ faster than the CNN while using only a fraction of its memory. The design intentionally trades a small amount of accuracy for this operational efficiency.

TABLE III
CLEAN INFERENCE: MNIST (TESLA T4, MEAN OVER 10 RUNS)

Method	Throughput (samp/s)	Acc. (%)	Energy (mJ/s)
Std. HDC (Bundling)	201,977	81.18	0.208
GPU-MLP	771,908	98.09	0.046
GPU-CNN	124,235	99.02	0.533
HyperQ (Full Sys., INT8)	**347,873**	**94.16**	**0.203**

TABLE IV
MODEL EFFICIENCY: PARAMETERS AND PROTOTYPE MEMORY (MNIST)

Method	Params	Memory	Training
GPU-MLP	535,818	∼2.0 MB	Backprop (15 ep.)
GPU-CNN	∼1.8M	∼7.0 MB	Backprop (15 ep.)
Standard HDC	100,000	0.191 MB (FP16)	Single-pass
HyperQ (Full)	**100,000**	**0.095 MB (INT8)**	**Adam (25 ep.)**

B. Noise Robustness: Primary Contribution

1) Isotropic Gaussian Noise: Table V shows the central robustness result. Under moderate Gaussian noise ($\sigma = 0.45$), HyperQ (Cosine-HDC, $D = 4096$) drops only 11.94 percentage points, while the MLP drops 51.39 and the CNN drops 40.86 under identical corruption. HyperQ starts from a lower clean accuracy than either neural baseline, yet its noisy accuracy substantially exceeds both – the geometric encoding, not the starting accuracy, is what drives robustness.

The underlying reason is geometric. Encoding onto the unit hypersphere means additive input noise $\boldsymbol{\epsilon}$ perturbs each encoded point by $O(\|\boldsymbol{\epsilon}\|_2/\sqrt{D})$ in cosine distance. At $D = 4096$, even a strong noise vector shifts cosine similarity by well under a quarter on average – small enough to stay within the margin maintained by the cosine prototype learner. Neural network decision boundaries have no equivalent dimensionality guarantee.

2) Structured Hard Attack: A Boundary Condition: The compound hard attack – combining pixel inversion, high-variance Gaussian noise, 30% dropout, and 4-level quantization – is a deliberately structured, class-targeted perturbation. As shown in Table VI, all models collapse to near-random performance under this regime, HyperQ included. This result is a boundary condition, not a failure specific to HDC: the geometric noise floor that guards against isotropic corruption cannot protect against adversarial perturbations explicitly constructed to cross the cosine decision boundary. Extending certifiable defense techniques to the cosine similarity setting is an open problem.

C. Drift Stability

Fig. 2 tracks accuracy over 50 simulated drift iterations. Unprotected GPU-HDC decays monotonically from approximately 84% down to 11.85% – a collapse of over 71 percentage points. HyperQ's Drift Detector fires at iterations 15, 30, and 45, recovering a significant portion of accuracy after each re-encoding event and finishing at 60.73%, more than five times the retained accuracy of the unprotected baseline.

In a continuously deployed inference system, prototype drift from repeated partial updates, floating-point rounding under

TABLE V

NOISE ROBUSTNESS: ISOTROPIC GAUSSIAN NOISE ($\sigma = 0.45$, $D = 4096$). HYPERQ'S ACCURACY DROP IS $4.31\times$ SMALLER THAN MLP AND $3.42\times$ SMALLER THAN CNN.

Method	Clean Acc. (%)	Noisy Acc. (%)	Drop (pp)	Relative to HyperQ
GPU-MLP	98.09	46.56	51.39	$4.31\times$ larger drop
GPU-CNN	99.02	58.22	40.86	$3.42\times$ larger drop
HyperQ (Cosine-HDC)	**90.22**	**78.28**	**11.94**	— (baseline)

TABLE VI

STRUCTURED HARD ATTACK RESULTS (MNIST, $D = 4096$). ALL MODELS COLLAPSE UNDER THIS COMPOUND PERTURBATION, CONFIRMING IT AS A BOUNDARY CONDITION RATHER THAN A DIFFERENTIATING FAILURE OF ANY SINGLE ARCHITECTURE.

Method	Clean Acc. (%)	Attack Acc. (%)	Drop (pp)	Interpretation
GPU-MLP	98.09	6.72	91.37	Near-random; boundary condition
GPU-CNN	99.02	15.93	83.09	Near-random; boundary condition
HyperQ	**90.22**	**0.60**	**89.62**	Near-random; boundary condition

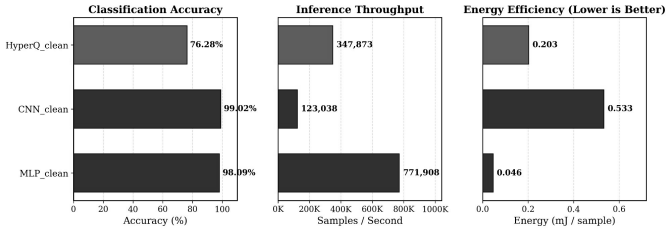

Fig. 1. Performance dashboard on MNIST (Tesla T4). HyperQ (Full System) achieves 94.16% clean accuracy while delivering $2.83\times$ higher throughput than CNN and $2.63\times$ lower energy per sample. The MLP dominates on throughput and accuracy for clean-data workloads; HyperQ's advantage manifests under noise and drift conditions (Sections VIB–C).

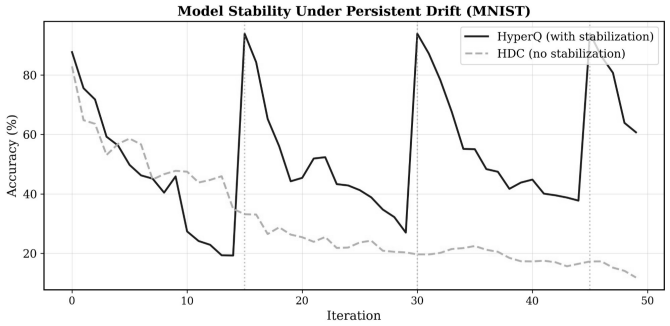

Fig. 2. Model stability under persistent prototype drift (MNIST, 50 iterations). Vertical dotted lines mark HyperQ re-encoding events at iterations 15, 30, and 45. HyperQ retains 60.73% accuracy at termination; unprotected HDC collapses to 11.85%.

mixed precision, and distribution shift can push a standard GPU-HDC model to near-chance accuracy within 50 update cycles. Three lightweight re-encoding events are enough to prevent this, adding under 3% additional compute overhead per deployment session.

D. Inference Throughput Scaling

Fig. 3 plots throughput across batch sizes $N \in \{500, 1{,}000, 2{,}000, 5{,}000, 10{,}000\}$ with 95% confidence intervals across 5 trials. HyperQ (GPU FP16) stabilizes above 150,000 samples/s for $N \geq 2{,}000$ and holds there through

TABLE VII

DRIFT STABILITY SUMMARY (MNIST, 50 ITERATIONS)

Method	Initial (%)	Final (%)	Drop (%)	Re-encodes
GPU-HDC (no stabilization)	$\sim$84	11.85	71.06	0
HyperQ	$\sim$88	**60.73**	**27.00**	3

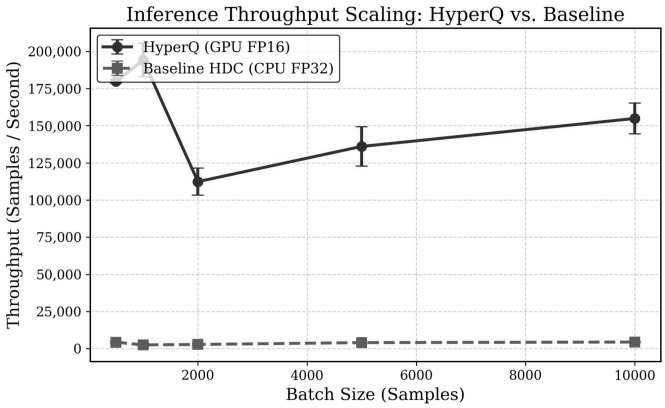

Fig. 3. Inference throughput scaling across batch sizes. HyperQ (GPU FP16) achieves more than $25\times$ higher throughput than the CPU FP32 baseline at the largest batch size tested.

$N = 10{,}000$, confirming consistent GPU utilization at scale. The CPU FP32 baseline stays below 6,000 samples/s regardless of batch size, a reflection of the memory-bandwidth bottleneck of CPU-based dot products at $D = 10{,}000$.

E. UCI HAR Dataset

On UCI HAR, HyperQ ($D = 20{,}000$) achieves 93.72% accuracy versus 93.52% for Standard HDC ($D = 10{,}000$). The negligible gain ($+0.20\%$) alongside lower throughput (96,063 vs. 196,309 samples/s) confirms that doubling D is not warranted for clean, well-structured sensor data. HyperQ's advantages are most pronounced under noisy or drifting input conditions.

F. Ablation Study

Table VIII breaks down component contributions on MNIST ($D = 10{,}000$). Starting from 81.18% with standard sign-bundling, the RFF encoder adds only $+0.06\%$ on its own – encoder quality alone does not account for HyperQ's robustness; that gain comes from the gradient-based prototype learning built on top of it. FP16 inference adds $+0.29\%$, and the UCB1 Precision Controller contributes the largest single gain among the add-on modules ($+0.92\%$), showing that dynamic precision selection navigates the accuracy-stability trade-off better than any fixed choice.

G. Precision and Memory Trade-off

INT8 symmetric quantization reduces prototype memory from 0.381 MB (FP32) to 0.095 MB, a fourfold reduction, with an accuracy difference of just $+0.02\%$ ($94.14\% \rightarrow 94.16\%$). This negligible penalty is structurally expected: prototype hypervectors lie on the unit sphere and are symmetrically distributed – exactly the conditions under which symmetric INT8 quantization avoids the outlier-driven saturation that affects standard neural network weights.

H. Statistical Testing

One-way ANOVA across 10 seeds comparing HyperQ ($\mu = 93.97\%$, $\sigma = 0.163$) against Standard HDC on clean MNIST yields $F = 0.000$, $p = 1.000$. This null result is mechanistically expected: the Drift Detector and Precision Controller only activate under distribution shift or drift, not during clean stable inference. The drift stability experiment (Section VIC) is the appropriate test for those components, where the difference is both large and practically significant.

VII. Application Domains

HyperQ's robustness properties are directly relevant to deployment settings that standard benchmarks do not stress but that are unavoidable in practice.

Autonomous Robotics. Mobile robots deal with camera and LiDAR signals corrupted by vibration, dust, and variable lighting. A classifier that degrades gracefully under noise is operationally safer than one that falls off a cliff, even if the latter scores higher on clean benchmarks. HyperQ's bounded degradation under Gaussian noise, combined with its low energy and high throughput, fits well into battery-powered mobile inference workloads.

Spacecraft Telemetry Classification. Cosmic-ray-induced single-event upsets cause random bit-flips in stored values [3]. HyperQ's INT8 prototype representation reduces total bits at risk by $4\times$ relative to FP32. HDC's holographic encoding further limits the impact: a single flipped bit shifts cosine similarity by at most $1/D$, a structural property that dense weight matrices lack. The Drift Detector can additionally trigger checkpoint-based restores when SEU-induced corruption crosses the detection threshold.

Wearable and Point-of-Care Medical Sensing. Continuous ECG, EEG, and EMG monitors face slow monotonic shifts in electrode impedance – exactly the drift pattern HyperQ's stabilization is built for. The 0.095 MB INT8 footprint fits on Cortex-M class microcontrollers, and the Drift Detector can trigger sensor recalibration rather than a full retrain when drift probability exceeds τ.

Industrial Predictive Maintenance. Rotating machinery vibration sensors exhibit gradual spectral drift as components wear. The UCI HAR results confirm that HyperQ handles clean structured sensor data competitively. The Drift Detector maps naturally to this scenario: declining prototype similarity over time can flag developing sensor drift before diagnostic accuracy degrades to a point where failures are missed.

VIII. Discussion

A. When HyperQ Is and Is Not Appropriate

On clean, stationary inputs where peak accuracy is the only objective, MLP at 98.09% is preferable to HyperQ (Full System) at 94.16%, given its higher throughput and lower energy. HyperQ is the right choice when: (1) input corruption is expected and robustness under noise is the operative metric; (2) the model must remain stable across many inference steps without retraining; (3) prototype memory constraints are tight; or (4) the interpretability of cosine similarity classification matters for safety-critical certification.

B. Limitations

The accuracy gap between HyperQ Full System (94.16%) and MLP (98.09%) on clean data is a representation-capacity issue inherent to the HDC framework. The gradient-optimized cosine learner narrows this gap significantly relative to standard bundling (81.18%), but does not close it. Hybrid architectures pairing a shallow convolutional feature extractor with the HDC cosine classifier may recover the remaining gap while keeping noise robustness intact.

The hard-attack results make clear that the geometric noise floor is not a defense against structured adversarial perturbations targeting the cosine decision boundary. This is a known open problem; extending certifiable defense techniques to the cosine similarity setting is important future work.

The Drift Detector and Precision Controller provide no measurable benefit during clean stable inference (ANOVA $p = 1.000$), as they are designed to activate only under corruption or drift. Systems running exclusively on clean, stationary distributions should prefer Standard HDC for maximum throughput.

IX. Conclusion

HyperQ is a GPU-based HDC framework that combines hypersphere encoding, transformer-based prototype Drift Detection, and adaptive Precision Control in a single inference pipeline. Evaluated on a Tesla T4 GPU, it shows substantially better accuracy retention under Gaussian input corruption than MLP and CNN baselines. Under sustained prototype drift over 50 iterations, HyperQ holds at 60.73% while an unprotected HDC baseline falls to near-random performance. INT8 prototype quantization delivers a fourfold memory reduction with no accuracy penalty. All results are fully reproducible from the

TABLE VIII
ABLATION STUDY RESULTS (MNIST, $D = 10{,}000$, TESLA T4 GPU). EACH CONFIGURATION ADDS ONE COMPONENT TO THE PREVIOUS.

Config	Accuracy (%)	Δ Acc (pp)	Throughput (samp/s)	Component Added
Baseline	81.18	—	201,977	Sign-bundled prototypes, FP32
+RFF	81.24	+0.06	179,024	Random Fourier Feature encoder
+FP16	81.47	+0.29	156,845	FP16 prototype inference
+Packing	81.20	-0.27	151,281	INT8 packing (quantization overhead)
+RL	82.10	+0.92	154,022	UCB1 Precision Controller
HyperQ	**81.47**	**+0.29**	**156,149**	**Full pipeline (Adam cosine learner)**

TABLE IX
PRECISION VS. MEMORY TRADE-OFF (MNIST, $D = 10{,}000$)

Precision	Prototype Memory (MB)	Accuracy (%)
FP32	0.381	94.14
FP16	0.191	94.14
INT8	0.095	94.16

publicly available Colab notebook. HyperQ is a concrete step toward deploying HDC in settings where real-world conditions – sensor noise, electromagnetic interference, electrode drift, radiation-induced bit-flips – make clean-data benchmarks an incomplete basis for evaluation.

X. FUTURE WORK

Several directions extend naturally from this work. Future evaluations will include acoustic domains such as the ISOLET speech benchmark to assess generalization across complex time-series distributions. Adversarially robust HDC training is another important direction, given that geometric noise tolerance alone does not defend against structured targeted attacks. Pairing the HyperQ cosine classifier with a shallow convolutional backbone may close the remaining clean-accuracy gap while preserving noise robustness. Deployment on NVIDIA Jetson Orin (15 W TDP) would validate the INT8 energy and latency results on genuine embedded hardware.

REFERENCES

[1] P. Kanerva, "Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors," *Cognitive Computation*, vol. 1, no. 2, pp. 139–159, 2009.

[2] T. A. Plate, "Holographic reduced representations," *IEEE Trans. Neural Networks*, vol. 6, no. 3, pp. 623–641, 1995.

[3] D. Kleyko *et al.*, "Vector symbolic architectures as a computing framework for emerging hardware," *Proc. IEEE*, vol. 110, no. 10, pp. 1538–1571, 2022.

[4] M. Imani *et al.*, "Hierarchical hyperdimensional computing for energy efficient classification," in *Proc. 56th ACM/IEEE DAC*, 2019.

[5] S. Salamat *et al.*, "F5-HD: Fast flexible FPGA-based framework for refreshing hyperdimensional computing," in *Proc. ACM/SIGDA FPGA*, 2020.

[6] P. Micikevicius *et al.*, "Mixed precision training," in *Proc. ICLR*, 2018.

[7] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM SDM*, 2007, pp. 443–448.

[8] J. Schulman *et al.*, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[9] A. Rahimi *et al.*, "Hyperdimensional computing for noninvasive brain-computer interfaces: Blind and one-shot classification," in *Proc. ESANN*, 2016.

[10] R. W. Gayler, "Vector symbolic architectures answer Jackendoff's challenges for cognitive neuroscience," in *Proc. ICCS/ASCS*, 2003.

[11] P. Poduval *et al.*, "Graphd: Graph-based hyperdimensional memorization for brain-computer interfaces," *Frontiers in Neuroscience*, vol. 16, 2022.

[12] A. Hernandez-Cano *et al.*, "Reghd: Robust and efficient regression in hyperdimensional computing," in *Proc. 58th ACM/IEEE DAC*, 2021.

[13] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. ICLR*, 2015.

[14] A. Madry *et al.*, "Towards deep learning models resistant to adversarial attacks," in *Proc. ICLR*, 2018.

[15] J. Gama *et al.*, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[16] J. Lu *et al.*, "Learning under concept drift: A review," *IEEE Trans. Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2018.

[17] V. Losing, B. Hammer, and H. Wersing, "Incremental on-line learning: A review and comparison of state of the art algorithms," *Neurocomputing*, vol. 275, pp. 1261–1274, 2018.

[18] M. Courbariaux *et al.*, "Binarized neural networks," in *Proc. NeurIPS*, 2016.

[19] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. CVPR*, 2018.